

Stage 2 Step-by-Step Practical & Conceptual Recap

1. In-Memory Data Structures (RAM Cache)

- **What We Built:** Created an array of 3 objects representing our task database stored directly in volatile memory (`let tasks = [...]`).
- **Sysadmin Analogy:** This is like loading a temporary lookup table into Windows Registry memory or an active RAM cache. When the Node process restarts, the data resets, exactly like volatile system cache clearing upon a server reboot.

2. Express Path Parameters (`/:id`)

- **What We Built:** Added `app.get('/tasks/:id', ...)` to extract dynamic IDs from the request URL path.
- **Sysadmin Analogy:** Think of the colon prefix (`:id`) as a wildcard variable or positional parameter in a PowerShell script (e.g., `Get-Service -Name $ServiceName`). It tells the routing firewall to capture whatever integer is passed in the URI string.

3. Defensive Programming & HTTP 404 Error Handling

- **What We Built:** Implemented a verification gate using `.find()` combined with an `if (!task)` check that explicitly returns an HTTP 404 Not Found status and a structured JSON error payload (`{ "error": "Task 99 not found" }`).
- **Sysadmin Analogy:** This is identical to querying Windows Event Viewer for a Record ID that doesn't exist (`Get-WinEvent -InstanceId 99`). Instead of returning an empty success code (200 OK), a robust system returns a strict 404 Not Found so automated monitoring tools and API clients immediately recognize that the record is missing.

4. Protocol Verification with `curl.exe`

- **What We Executed:** Ran `curl.exe -i http://localhost:3000/tasks/1` and `curl.exe -i http://localhost:3000/tasks/99` in the VS Code terminal.
- **Sysadmin Analogy:** Using `curl.exe` with the `-i` flag inspects raw HTTP response headers and status codes, acting like a network packet analyzer (Wireshark or Test-NetConnection) to verify that your API communication layer adheres strictly to web standards.

5. Version Control Checkpoint

- **What We Executed:** Staged and committed our changes with `git add .` and `git commit -m "stage2: read endpoints with 404"`.
- **Sysadmin Analogy:** This creates an immutable system restore point, documenting your exact configuration state before moving on to write creation and update endpoints in Stage

3.